

Министерство образования Российской Федерации
МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
им. Н.Э. БАУМАНА

Факультет: Информатика и системы управления
Кафедра: Информационная безопасность (ИУ8)

ТЕОРИЯ ИГР В СИСТЕМАХ ЗАЩИТЫ
ИНФОРМАЦИИ

Лабораторная работа №2 на тему:
«Решение матричных игр с нулевой суммой
аналитическим (матричным) и числовым (Брауна-
Робинсона) методами»

Вариант 13

Преподаватель:
Захаров П.В.

Студент:
Степанова Е.А.

Группа:
ИУ8-21М

Москва 2026

Цель работы

Изучить аналитический (метод обратной матрицы) и численный (метод Брауна-Робинсона) подходы [1] к нахождению смешанных стратегий в антагонистической игре двух лиц в нормальной форме.

Постановка задачи

Решите приведенную в варианте задания игру (найдите цену игры и оптимальные стратегии обоих игроков) методами обратной матрицы и Брауна-Робинсона. Сравните полученные результаты.

Ход работы

Пусть (3×3) – игра Γ задана матрицей:

$$C = \begin{pmatrix} 9 & 10 & 13 \\ 1 & 18 & 11 \\ 17 & 4 & 0 \end{pmatrix}$$

Аналитический метод.

Теорема 1. Вполне смешанная игра $(m \times n)$ – игра Γ имеет единственную ситуацию равновесия (x^*, y^*) и квадратичную матрицу $(m = n)$;

если цена игры $v \neq 0$, то матрица C невырожденная и

$$x^* = \frac{C^{-1}u^T}{u C^{-1}u^T}, \quad y^* = \frac{uC^{-1}}{u C^{-1}u^T}, \quad v = \frac{1}{u C^{-1}u^T} \quad (1)$$

Расчёт по формулам (1) дает следующее аналитическое решение задачи, рисунок 1:

$$\begin{aligned} x^* &= (0.630952, \quad 0.138889, \quad 0.230159) \\ y^* &= (0.460317, \quad 0.47619, \quad 0.063492) \\ v &= 9.730159 \end{aligned}$$

Аналитическое решение (обратная матрица)

Цена игры $v = 9.730159$

Стратегия $X^* = [0.630952 \ 0.138889 \ 0.230159]$

Стратегия $Y^* = [0.460317 \ 0.47619 \ 0.063492]$

Рисунок 1. Результат расчета аналитическим методом на Python

Реализуем теперь алгоритм Брауна-Робинсона. Пусть на первом шаге игроки выбрали стратегии x_1, y_1 . Учитывая, что игрок А выбрал x_1 , игрок В мог получить один из выигрышей (9, 10, 13). А если игрок В выбрал y_1 , то возможные выигрыши игрока А были (9, 1, 17). Следовательно, на втором этапе игрокам следует выбирать стратегии x_2 и y_2 , соответственно.

Результаты расчетов для первых 4 шагов приведены в таблице 1.

Таким образом, за 4 шага получены следующие приближенные смешанные стратегии:

$$\tilde{x}[4] = \left(\frac{1}{4}, 0, \frac{3}{4}\right), \tilde{y}[4] = \left(\frac{1}{2}, 0, \frac{1}{2}\right),$$

А погрешность равна

$$\varepsilon[4] = 7.75$$

Таблица 1. Первые шаги алгоритма Брауна-Робинсона для примера

№пп	Выбор А	Выбор В	Выигрыш А			Выигрыш В			$\frac{1}{k} \bar{v}[k]$	$\frac{1}{k} \underline{v}[k]$
			x_1	x_2	x_3	y_1	y_2	y_3		
1	x_1	y_1	9	1	17	9	10	13	17	9
2	x_3	y_1	9	1	17	13	7	6.5	17	6.5
3	x_3	y_3	10.33	4.33	11.33	14.33	6	4.33	11.333	4.33
4	x_3	y_3	11	6	8.5	15	5.50	3.25	11	3.25

Результаты методом Брауна-Робинсона были получены через 7310 итераций, рис.2.

```
Итераций: 7310  
Цена игры  $\approx 9.7493160055$   
 $A^* \approx (0.637073, 0.134063, 0.228865)$   
 $B^* \approx (0.451026, 0.465527, 0.083447)$ 
```

Рисунок 2. Результат расчета методом Брауна-Робинсона на Python

Метод Брауна-Робинсона применим для матричных игр, но требует большого числа итераций для достижения приемлемой точности. Аналитический метод универсален и предпочтителен для игр малой размерности.

Приложение А.

Файл 'T12.py'.

```
'''
import numpy as np

C = np.array([
    [9, 10, 13],
    [1, 18, 11],
    [17, 4, 0]
], dtype=float)

# Аналитическое решение
def analytical_solution(C):
    m, n = C.shape
    if m != n:
        raise ValueError("Матрица должна быть квадратной")
    C_inv = np.linalg.inv(C)
    u = np.ones(m)
    denom = u @ C_inv @ u
    if denom == 0:
        raise ValueError("Цена игры равна нулю")
    v = 1.0 / denom
    x = (u @ C_inv) / denom
    y = (C_inv @ u) / denom
    return x, y, v

x_anal, y_anal, v_anal = analytical_solution(C)

print("АНАЛИТИЧЕСКОЕ РЕШЕНИЕ")
print(f"Цена игры = {v_anal:.10f}")
print(f"A* = ({x_anal[0]:.6f}, {x_anal[1]:.6f}, {x_anal[2]:.6f})")
print(f"B* = ({y_anal[0]:.6f}, {y_anal[1]:.6f}, {y_anal[2]:.6f})")
print()

# Метод Брауна-Робинсона
def brown_robinson_auto(C, eps=0.1, report_interval=1000):
    m, n = C.shape
    freqA = np.zeros(m, dtype=int)
    freqB = np.zeros(n, dtype=int)
    a, b = 0, 0
    print("="*110)
    print(f"{'№ пп':>4} | {'Выбор А':>7} | {'Выбор В':>7} | {'Выигрыш А  
(x1,x2,x3)':<25} | {'Выигрыш В (y1,y2,y3)':<25} | {'v_нижн':>8} |  
{'v_верхн':>8}")
    print("-"*110)
    k = 0
    while True:
        k += 1
```

```

    freqA[a] += 1
    freqB[b] += 1
    x_emp = freqA / k
    y_emp = freqB / k
    expA = C @ y_emp
    expB = x_emp @ C
    lower = np.max(expA)
    upper = np.min(expB)
    if k <= 20:
        A_str = f"({expA[0]:5.2f}, {expA[1]:5.2f}, {expA[2]:5.2f})"
        B_str = f"({expB[0]:5.2f}, {expB[1]:5.2f}, {expB[2]:5.2f})"
        print(f"{k:4d} | {a+1:3d} | {b+1:3d} | {A_str:25} | {B_str:25} | {lower:8.4f} | {upper:8.4f}")
    if k > 20 and k % report_interval == 0:
        print(f"Шаг {k:8d}: нижняя = {lower:10.4f}, верхняя = {upper:10.4f}, разница = {abs(upper-lower):10.6f}")
        if abs(upper - lower) <= eps:
            print("-"*110)
            print(f"Сходимость на шаге {k},  $\varepsilon$  = {abs(upper-lower):.6f}")
            break
    a = np.argmax(expA)
    b = np.argmin(expB)
    x_final = freqA / k
    y_final = freqB / k
    v_final = (lower + upper) / 2
    return x_final, y_final, v_final, k

print("\nЧИСЛЕННЫЙ МЕТОД ( $\varepsilon=0.1$ )")
x_num, y_num, v_num, steps = brown_robinson_auto(C, eps=0.1,
report_interval=5000)

#
print("\nМетод Брауна-Робинсона")
print(f"Итераций: {steps}")
print(f"Цена игры  $\approx$  {v_num:.10f}")
print(f"A*  $\approx$  ({x_num[0]:.6f}, {x_num[1]:.6f}, {x_num[2]:.6f})")
print(f"B*  $\approx$  ({y_num[0]:.6f}, {y_num[1]:.6f}, {y_num[2]:.6f})")

print("\nСПРАВНЕНИЕ")
print(f"Погрешность по цене: {abs(v_anal - v_num):.8f}")
print(f"Погрешность A: max = {np.max(np.abs(x_anal - x_num)):.8f}")
print(f"Погрешность B: max = {np.max(np.abs(y_anal - y_num)):.8f}")

```